AWS CLI

1. ## What is AWS CLI?

   - The **AWS Command Line Interface (AWS CLI)** is an open-source tool that allows you to manage and interact with **Amazon Web Services(AWS)** directly from our terminal.
   - Instead of clicking through buttons in a browser (the **AWS Management Console**), you type text-based commands to control your cloud resources.

2. ## What are the Core Features of AWS CLI?

   - **Unified Tool**: You only need this one tool to manage hundreds of different AWS services, such as launching Amazon EC2 instances or managing Amazon S3 buckets.
   - **Automation**: Because it runs in a terminal, it is ideal for writing **scripts** (like Bash or PowerShell) to automate repetitive tasks.
   - **Cross-Platform**: It works on **Windows, macOS, and Linux**.
   - **Direct API Access**: It provides direct access to the public APIs of AWS services, offering functionality equivalent to the web console.

3. ## Why do we need to use It?

   - **Speed & Efficiency**: Once you learn the commands, it is often faster to type a single line than to navigate through multiple web pages.
   - **Resource Management**: You can manage resources at scale, such as stopping 50 servers at once with a single script.
   - **Reproducibility**: Scripts ensure that infrastructure is set up exactly the same way every time.

4. ## How It Works?

   - **Installation**: You download the AWS CLI installer for your operating system.
   - **Configuration**: You run `aws configure` to provide your **Access Key ID** and **Secret Access Key**. (**Not Recommended**)
   - **IAM Role Configuration:** The AWS CLI automatically inherits permissions from the attached **IAM Role**, requiring no manual configuration or access keys. If the **server** is from the **AWS** then this method is **Recommended**.
   - **Execution**: You run commands following this structure: `aws <service> <operation> <parameters>`.
     - *Example to list S3 buckets:* `aws s3 ls`.

Private Network in AWS

1. **What is AWS VPC?**
    - A Virtual Private Cloud (VPC) is a logically isolated section of the cloud dedicated to your account. It is your own private network where you define the IP address range and have full control over virtual networking resources.
    - **Function:** It provides complete control over your virtual networking environment, including your own IP address range (CIDR block).
    - **Scope:** A VPC is specific to one AWS Region but spans across all Availability Zones (AZs) within that region.
2. **What is a Subnet in VPC?**
    - A **Subnet** is a range of IP addresses in your VPC. You launch resources, like EC2 instances, into specific subnets to group them based on security or operational needs.
    - Subnets are subdivisions of your VPC's IP range. They are used to group resources based on security and operational needs.
    - **Public Subnet:** Used for resources that must be accessible from the internet, like web servers.
    - **Private Subnet:** Used for resources that should never be directly reached from the internet, such as databases.
    - **Scope:** Each subnet resides entirely within one Availability Zone in the Region.
3. **What is a Route Table in VPC?**
    - A **Route Table** contains a set of rules (called routes) that determine where network traffic from your subnet or gateway is directed. Every subnet must be associated with one route table.
    - **Local Route:** Every VPC has a default "local" route that allows all subnets within that VPC to talk to each other.
    - **Custom Routes:** You add custom routes to send traffic to gateways (like an IGW for the internet).
4. **What is IGW in VPC?**
    - The **Internet Gateway(IGW)** is a redundant, highly available VPC component that allows communication between your VPC and the internet. It provides a target in your route tables for internet-bound traffic.
    - It is a horizontally scaled, redundant component that connects your VPC to the public internet.
    - **How it works:** To make a subnet "public," you must add a route in its Route Table that sends internet-bound traffic (0.0.0.0/0) to the IGW.
    - **Relationship:** You can attach only one IGW to a VPC at a time.

5. **What is Network Security Group(NSG) in VPC?**
    - Often referred to simply as a **Security Group**, this acts as a virtual firewall for your **instances** (like EC2).
    - **Stateful:** If you allow an incoming request, the response is automatically allowed back out, even if no outbound rule exists.
    - **Granular:** You can assign different security groups to different instances within the same subnet.
    - **Rules:** You can only specify "Allow" rules; everything else is denied by default.
6. **What is NACL in VPC?**
    - A Network ACL is an optional layer of security that acts as a firewall for controlling traffic in and out of one or more **subnets**.
    - **Stateless:** It does not remember sessions. If you allow traffic in, you must also explicitly create a rule to allow it back out.
    - **Order Matters:** Rules are evaluated in numerical order (lowest to highest). Once a match is found, it is applied, and no further rules are checked.
    - **Rules:** Unlike Security Groups, NACLs support both **Allow** and **Deny** rules (useful for blacklisting specific IP addresses).
7. **Key Differences: Security Groups vs. NACLs**

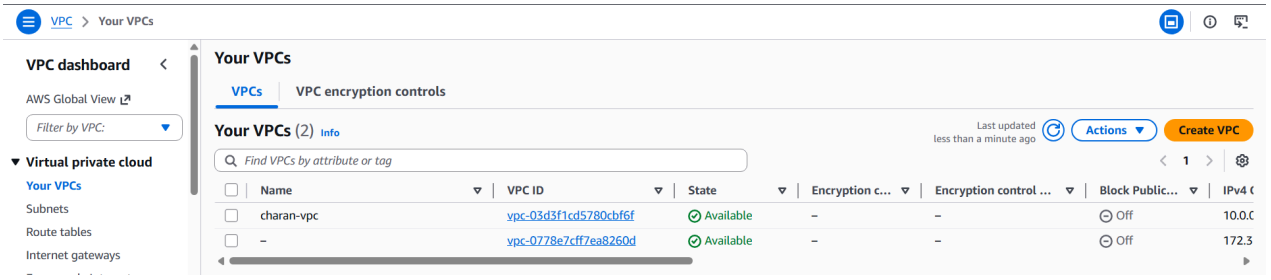
Feature	Security Group (NSG)	Network ACL (NACL)
Layer	Second layer of defense (Instance level)	First layer of defense (Subnet level)
State	**Stateful** (Remembers connections)	**Stateless** (Inbound/Outbound rules separate)
Rules	Supports **Allow** rules only	Supports **Allow** and **Deny** rules
Evaluation	All rules evaluated before traffic is allowed	Rules evaluated in order by rule number

Creation of VPC in AWS using CLI

Steps to Create via AWS CLI:

- Follow these steps in order. Replace placeholders like vpc-id with the actual IDs returned by your terminal.

1. **Create the VPC:**
    - `aws ec2 create-vpc --cidr-block 10.0.0.0/16 --tag-specifications "ResourceType=vpc,Tags=[{Key=Name,Value=charan-vpc}]"`



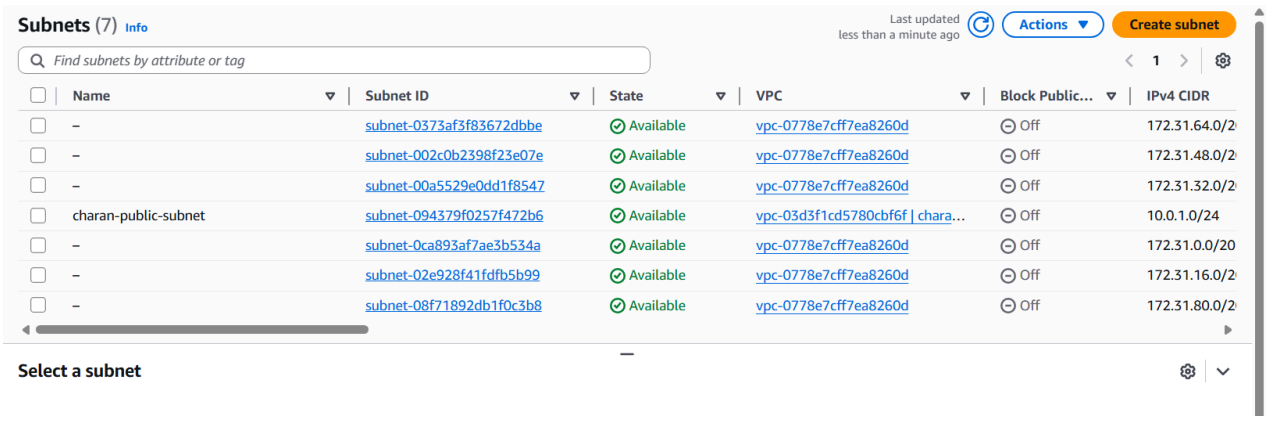

```
ubuntu@ip-172-31-39-155:~$ aws ec2 create-vpc --cidr-block 10.0.0.0/16 --tag-specifications "ResourceType=vpc,Tags=[{Key=Name,Value=charan-vpc}]"
{
    "Vpc": {
        "OwnerId": "381492023386",
        "InstanceTenancy": "default",
        "Ipv6CidrBlockAssociationSet": [],
        "CidrBlockAssociationSet": [
            {
                "AssociationId": "vpc-cidr-assoc-006d22b953047751a",
                "CidrBlock": "10.0.0.0/16",
                "CidrBlockState": {
                    "State": "associated"
                }
            }
        ],
        "IsDefault": false,
        "Tags": [
            {
                "Key": "Name",
                "Value": "charan-vpc"
            }
        ],
        "VpcId": "vpc-03d3f1cd5780cbf6f",
        "State": "pending",
        "CidrBlock": "10.0.0.0/16",
        "DhcpOptionsId": "dopt-0c89f46b55c35aff7"
    }
}
```

2. ## Create Subnets (Public and Private):

Public Subnet Creation:

`aws ec2 create-subnet --vpc-id vpc-id --cidr-block 10.0.1.0/24 --tag-specifications "ResourceType=subnet,Tags=[{Key=Name,Value=charan-public-subnet}]"`

```
ubuntu@ip-172-31-39-155:~$ aws ec2 create-subnet --vpc-id vpc-03d3f1cd5780cbf6f --cidr-block 10.0.1.0/24 --tag-specifications "ResourceType=subnet,Tags=[{Ke
y=Name,Value=charan-public-subnet}]"
{
    "Subnet": {
        "AvailabilityZoneId": "use1-az1",
        "MapCustomerOwnedIpOnLaunch": false,
        "OwnerId": "381492023386",
        "AssignIpv6AddressOnCreation": false,
        "Ipv6CidrBlockAssociationSet": [],
        "Tags": [
            {
                "Key": "Name",
                "Value": "charan-public-subnet"
            }
        ],
        "SubnetArn": "arn:aws:ec2:us-east-1:381492023386:subnet/subnet-094379f0257f472b6",
        "EnableDns64": false,
        "Ipv6Native": false,
        "PrivateDnsNameOptionsOnLaunch": {
            "HostnameType": "ip-name",
            "EnableResourceNameDnsARecord": false,
            "EnableResourceNameDnsAAAARecord": false
        },
        "SubnetId": "subnet-094379f0257f472b6",
        "State": "available",
        "VpcId": "vpc-03d3f1cd5780cbf6f",
        "CidrBlock": "10.0.1.0/24",
        "AvailableIpAddressCount": 251,
        "AvailabilityZone": "us-east-1c",
        "DefaultForAz": false,
        "MapPublicIpOnLaunch": false
    }
}
```



Private Subnet Creation:

`aws ec2 create-subnet --vpc-id vpc-id --cidr-block 10.0.2.0/24 --tag-specifications`

```
"ResourceType=subnet,Tags=[{Key=Name,Value=charan-private-subnet}]"
```

```
ubuntu@ip-172-31-39-155:~$ aws ec2 create-subnet --vpc-id vpc-03d3f1cd5780cbf6f --cidr-block 10.0.2.0/24 --tag-specifications "ResourceType=subnet,Tags=[{Ke
y=Name,Value=charan-private-subnet}]"
{
    "Subnet": {
        "AvailabilityZoneId": "use1-az1",
        "MapCustomerOwnedIpOnLaunch": false,
        "OwnerId": "381492023386",
        "AssignIpv6AddressOnCreation": false,
        "Ipv6CidrBlockAssociationSet": [],
        "Tags": [
            {
                "Key": "Name",
                "Value": "charan-private-subnet"
            }
        ],
        "SubnetArn": "arn:aws:ec2:us-east-1:381492023386:subnet/subnet-0853f3e3d117a36fe",
        "EnableDns64": false,
        "Ipv6Native": false,
        "PrivateDnsNameOptionsOnLaunch": {
            "HostnameType": "ip-name",
            "EnableResourceNameDnsARecord": false,
            "EnableResourceNameDnsAAAARecord": false
        },
        "SubnetId": "subnet-0853f3e3d117a36fe",
        "State": "available",
        "VpcId": "vpc-03d3f1cd5780cbf6f",
        "CidrBlock": "10.0.2.0/24",
        "AvailableIpAddressCount": 251,
        "AvailabilityZone": "us-east-1c",
        "DefaultForAz": false,
        "MapPublicIpOnLaunch": false
    }
}
```

Subnets (8) Info

Last updated less than a minute ago | Actions | Create subnet

Find subnets by attribute or tag

Name	Subnet ID	State	VPC	Block Public...	IPv4 CIDR
charan-private-subnet	subnet-0853f3e3d117a36fe	Available	vpc-03d3f1cd5780cbf6f | chara...	Off	10.0.2.0/24
–	subnet-0373af3f83672dbbe	Available	vpc-0778e7cff7ea8260d	Off	172.31.64.0/2
–	subnet-002c0b2398f23e07e	Available	vpc-0778e7cff7ea8260d	Off	172.31.48.0/2
–	subnet-00a5529e0dd1f8547	Available	vpc-0778e7cff7ea8260d	Off	172.31.32.0/2
charan-public-subnet	subnet-094379f0257f472b6	Available	vpc-03d3f1cd5780cbf6f | chara...	Off	10.0.1.0/24
–	subnet-0ca893af7ae3b534a	Available	vpc-0778e7cff7ea8260d	Off	172.31.0.0/20
–	subnet-02e928f41fdfb5b99	Available	vpc-0778e7cff7ea8260d	Off	172.31.16.0/2

Select a subnet

3. Set up the Internet Gateway (IGW):

```
aws ec2 create-internet-gateway --tag-specifications "ResourceType=internet-gateway,Tags=
[{Key=Name,Value=charan-igw}]"
```

```
ubuntu@ip-172-31-39-155:~$ aws ec2 create-internet-gateway --tag-specifications "ResourceType=internet-gateway,Tags=[{Key=Name,Value=charan-igw}]"
{
    "InternetGateway": {
        "Attachments": [],
        "InternetGatewayId": "igw-0f71d8bf8fa0ff3c4",
        "OwnerId": "381492023386",
        "Tags": [
            {
                "Key": "Name",
                "Value": "charan-igw"
            }
        ]
    }
}
```

Internet gateways (2) Info

Actions | Create internet gateway

Find internet gateways by attribute or tag

Name	Internet gateway ID	State	VPC ID	Owner
–	igw-0d204eb4f93462b2e	Attached	vpc-0778e7cff7ea8260d	381492023386
charan-igw	igw-0f71d8bf8fa0ff3c4	Detached	–	381492023386

To attach IGW to VPC:

```
aws ec2 attach-internet-gateway --vpc-id vpc-id --internet-gateway-id igw-id
```

```
ubuntu@ip-172-31-39-155:~$ aws ec2 attach-internet-gateway --vpc-id vpc-03d3f1cd5780cbf6f --internet-gateway-id igw-0f71d8bf8fa0ff3c4
ubuntu@ip-172-31-39-155:~$
```

Internet gateways (2) Info

Actions | Create internet gateway

Find internet gateways by attribute or tag

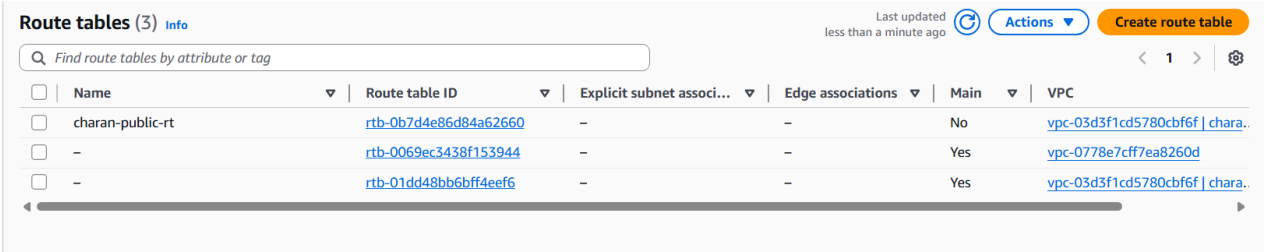
Name	Internet gateway ID	State	VPC ID	Owner
–	igw-0d204eb4f93462b2e	Attached	vpc-0778e7cff7ea8260d	381492023386
charan-igw	igw-0f71d8bf8fa0ff3c4	Attached	vpc-03d3f1cd5780cbf6f | charan-vpc	381492023386

4. **Configure Route Table:**

Public Route table:

`aws ec2 create-route-table --vpc-id vpc-id --tag-specifications "ResourceType=route-table,Tags=[{Key=Name,Value=charan-public-rt}]"`

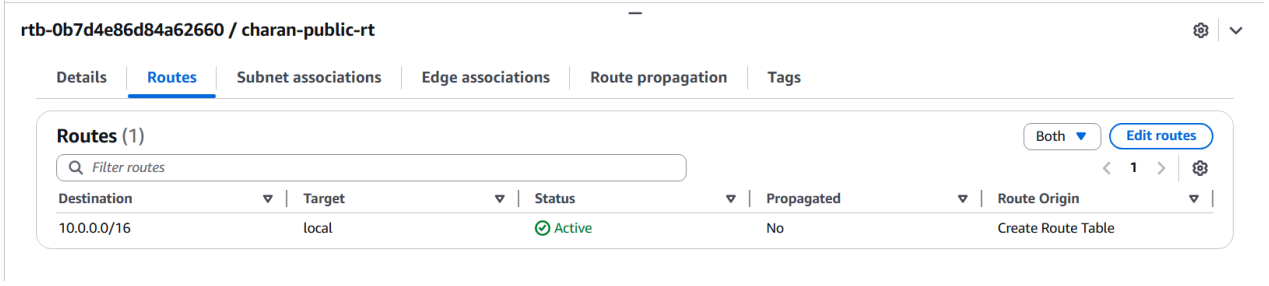
```
ubuntu@ip-172-31-39-155:~$ aws ec2 create-route-table --vpc-id vpc-03d3f1cd5780cbf6f --tag-specifications "ResourceType=route-table,Ta
gs=[{Key=Name,Value=charan-public-rt}]"
{
    "RouteTable": {
        "Associations": [],
        "PropagatingVgws": [],
        "RouteTableId": "rtb-0b7d4e86d84a62660",
        "Routes": [
            {
                "DestinationCidrBlock": "10.0.0.0/16",
                "GatewayId": "local",
                "Origin": "CreateRouteTable",
                "State": "active"
            }
        ],
        "Tags": [
            {
                "Key": "Name",
                "Value": "charan-public-rt"
            }
        ],
        "VpcId": "vpc-03d3f1cd5780cbf6f",
        "OwnerId": "381492023386"
    },
    "ClientToken": "998fa31f-bf8b-49bc-883e-9272816fe667"
}
ubuntu@ip-172-31-39-155:~$
```



Make it "Public" (Add Internet Route):

`aws ec2 create-route --route-table-id rtb-id --destination-cidr-block 0.0.0.0/0 --gateway-id igw-id`

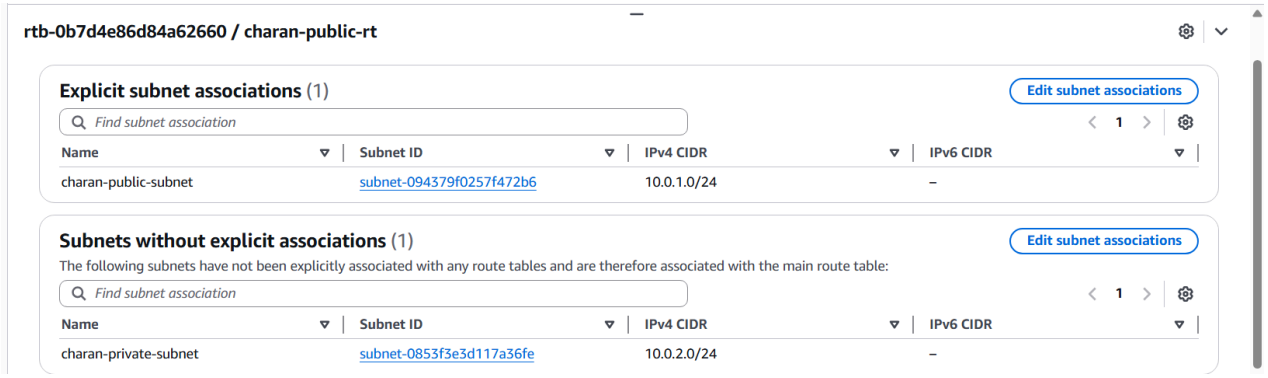
```
ubuntu@ip-172-31-39-155:~$ aws ec2 create-route --route-table-id rtb-0b7d4e86d84a62660 --destination-cidr-block 0.0.0.0/0 --gateway-id igw-0f71d8bf8fa0ff3c4
{
    "Return": true
}
ubuntu@ip-172-31-39-155:~$
```



Attach it to your Public Subnet:

`aws ec2 associate-route-table --subnet-id subnet-id --route-table-id rtb-id`

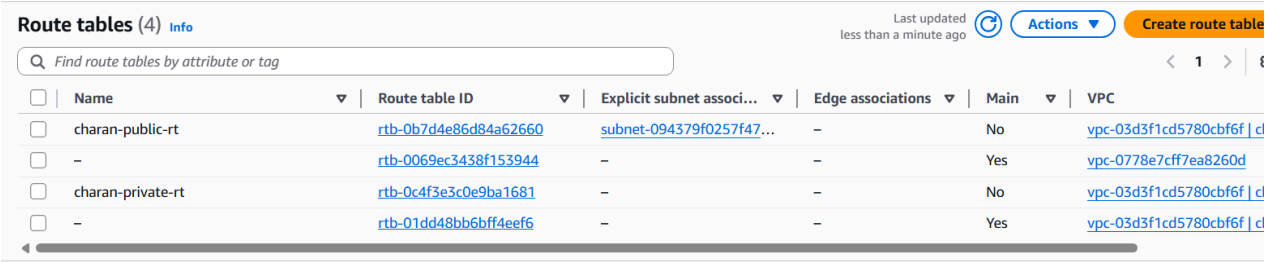
```
ubuntu@ip-172-31-39-155:~$ aws ec2 associate-route-table --subnet-id subnet-094379f0257f472b6 --route-table-id rtb-0b7d4e86d84a62660
{
    "AssociationId": "rtbassoc-0b2951b8fecca9d7a",
    "AssociationState": {
        "State": "associated"
    }
}
ubuntu@ip-172-31-39-155:~$
```



Private Route Table

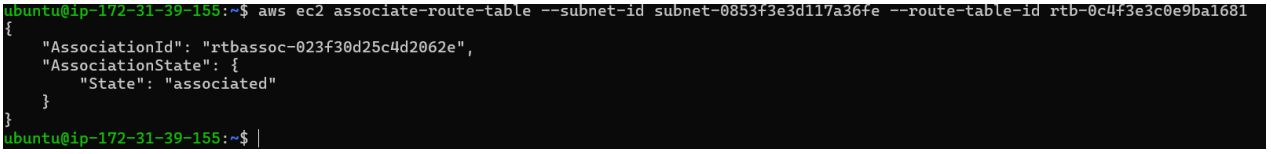
`aws ec2 create-route-table --vpc-id vpc-id --tag-specifications "ResourceType=route-table,Tags=[{Key=Name,Value=charan-private-rt}]"`

```
ubuntu@ip-172-31-39-155:~$ aws ec2 create-route-table --vpc-id vpc-03d3f1cd5780cbf6f --tag-specifications "ResourceType=route-table,Tags=[{Key=Name,Value=c
aran-private-rt}]"
{
    "RouteTable": {
        "Associations": [],
        "PropagatingVgws": [],
        "RouteTableId": "rtb-0c4f3e3c0e9ba1681",
        "Routes": [
            {
                "DestinationCidrBlock": "10.0.0.0/16",
                "GatewayId": "local",
                "Origin": "CreateRouteTable",
                "State": "active"
            }
        ],
        "Tags": [
            {
                "Key": "Name",
                "Value": "charan-private-rt"
            }
        ],
        "VpcId": "vpc-03d3f1cd5780cbf6f",
        "OwnerId": "381492023386"
    },
    "ClientToken": "70747ac7-fc18-4018-a885-cf5f8cf42e6a"
}
```

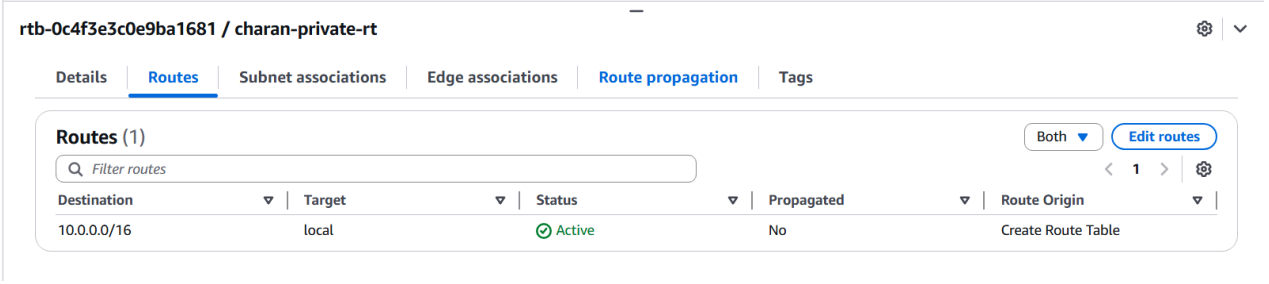


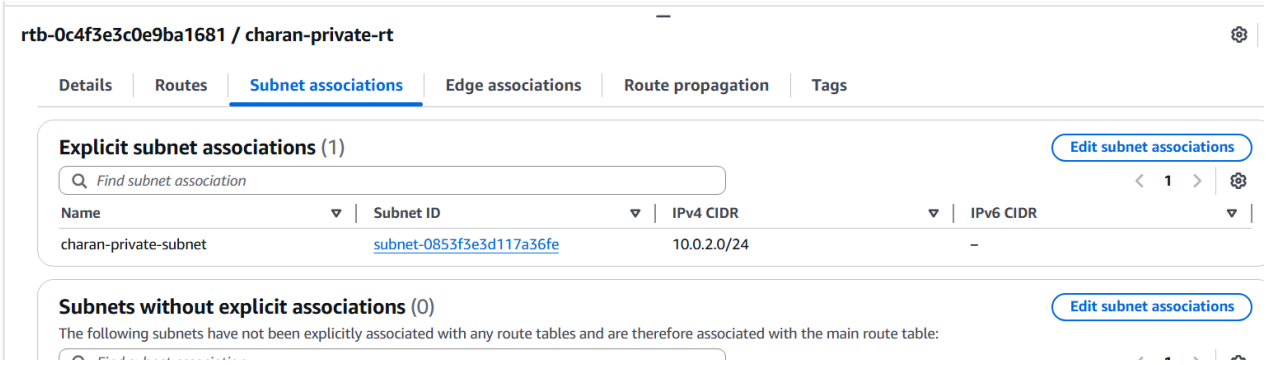
Attach it to your Private Subnet:

`aws ec2 associate-route-table --subnet-id subnet-id --route-table-id rtb-id`



```
ubuntu@ip-172-31-39-155:~$ aws ec2 associate-route-table --subnet-id subnet-0853f3e3d117a36fe --route-table-id rtb-0c4f3e3c0e9ba1681
{
    "AssociationId": "rtbassoc-023f30d25c4d2062e",
    "AssociationState": {
        "State": "associated"
    }
}
ubuntu@ip-172-31-39-155:~$
```

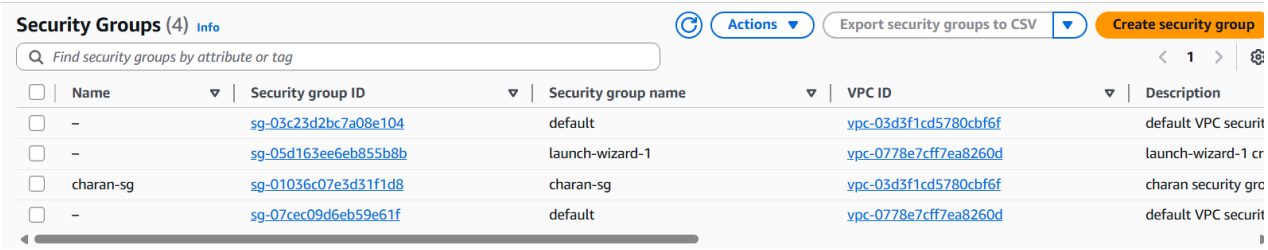




5. ## Create Security Group and Rules (SSH & HTTP):

`aws ec2 create-security-group --group-name charan-sg --description "charan security group" --vpc-id vpc-id --tag-specifications "ResourceType=security-group,Tags=[{Key=Name,Value=charan-sg}]"`

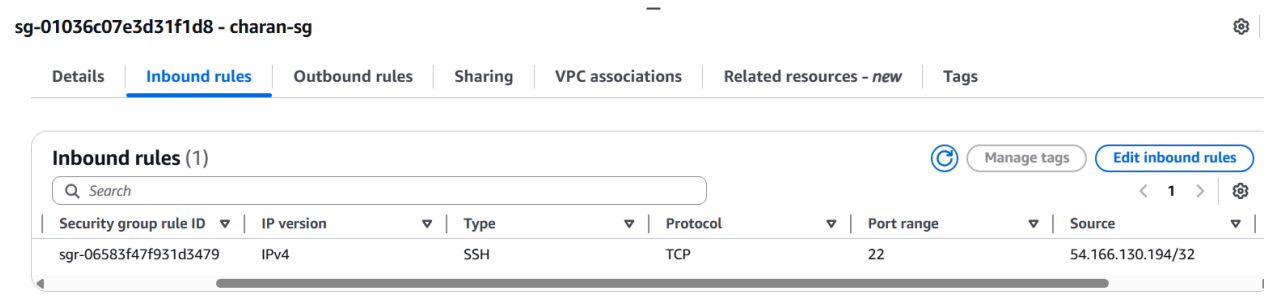
```
ubuntu@ip-172-31-39-155:~$ aws ec2 create-security-group --group-name charan-sg --description "charan security group" --vpc-id vpc-03d3f1cd5780cbf6f --tag-s
pecifications "ResourceType=security-group,Tags=[{Key=Name,Value=charan-sg}]"
{
    "GroupId": "sg-01036c07e3d31f1d8",
    "Tags": [
        {
            "Key": "Name",
            "Value": "charan-sg"
        }
    ],
    "SecurityGroupArn": "arn:aws:ec2:us-east-1:381492023386:security-group/sg-01036c07e3d31f1d8"
}
ubuntu@ip-172-31-39-155:~$
```



Open SSH (Port 22):

`aws ec2 authorize-security-group-ingress --group-id sg-id --protocol tcp --port 22 --cidr $(curl -s http://checkip.amazonaws.com)/32`

```
ubuntu@ip-172-31-39-155:~$ aws ec2 authorize-security-group-ingress --group-id sg-01036c07e3d31f1d8 --protocol tcp --port 22 --cidr $(curl -s http://checkip
.amazonaws.com)/32
{
    "Return": true,
    "SecurityGroupRules": [
        {
            "SecurityGroupRuleId": "sgr-06583f47f931d3479",
            "GroupId": "sg-01036c07e3d31f1d8",
            "GroupOwnerId": "381492023386",
            "IsEgress": false,
            "IpProtocol": "tcp",
            "FromPort": 22,
            "ToPort": 22,
            "CidrIpv4": "54.166.130.194/32",
            "SecurityGroupRuleArn": "arn:aws:ec2:us-east-1:381492023386:security-group-rule/sgr-06583f47f931d3479"
        }
    ]
}
ubuntu@ip-172-31-39-155:~$
```



Open HTTP (Port 80):

`aws ec2 authorize-security-group-ingress --group-id sg-id --protocol tcp --port 80 --cidr 0.0.0.0/0`

```
ubuntu@ip-172-31-39-155:~$ aws ec2 authorize-security-group-ingress --group-id sg-01036c07e3d31f1d8 --protocol tcp --port 80 --cidr 0.0.0.0/0
{
    "Return": true,
    "SecurityGroupRules": [
        {
            "SecurityGroupRuleId": "sgr-03d6580dc56ed7e8a",
            "GroupId": "sg-01036c07e3d31f1d8",
            "GroupOwnerId": "381492023386",
            "IsEgress": false,
            "IpProtocol": "tcp",
            "FromPort": 80,
            "ToPort": 80,
            "CidrIpv4": "0.0.0.0/0",
            "SecurityGroupRuleArn": "arn:aws:ec2:us-east-1:381492023386:security-group-rule/sgr-03d6580dc56ed7e8a"
        }
    ]
}
ubuntu@ip-172-31-39-155:~$
```

sg-01036c07e3d31f1d8 - charan-sg

Details | Inbound rules | Outbound rules | Sharing | VPC associations | Related resources - *new* | Tags

Inbound rules (2)

Manage tags | Edit inbound rules

Search

Name	Security group rule ID	IP version	Type	Protocol	Port range
–	sgr-03d6580dc56ed7e8a	IPv4	HTTP	TCP	80
–	sgr-06583f47f931d3479	IPv4	SSH	TCP	22

6. Create NACL:

VPCs come with a default NACL that allows all traffic. To create a custom one:*

`aws ec2 create-network-acl --vpc-id vpc-id --tag-specifications "ResourceType=network-acl,Tags=[{Key=Name,Value=charan-nacl}]"`

```
ubuntu@ip-172-31-39-155:~$ aws ec2 create-network-acl --vpc-id vpc-03d3f1cd5780cbf6f --tag-specifications "ResourceType=network-acl,Tags=[{Key=Name,Value=ch
aran-nacl}]"
{
    "NetworkAcl": {
        "Associations": [],
        "Entries": [
            {
                "CidrBlock": "0.0.0.0/0",
                "Egress": true,
                "IcmpTypeCode": {},
                "PortRange": {},
                "Protocol": "-1",
                "RuleAction": "deny",
                "RuleNumber": 32767
            },
            {
                "CidrBlock": "0.0.0.0/0",
                "Egress": false,
                "IcmpTypeCode": {},
                "PortRange": {},
                "Protocol": "-1",
                "RuleAction": "deny",
                "RuleNumber": 32767
            }
        ],
        "IsDefault": false,
        "NetworkAclId": "acl-05bcd5635092a317d",
        "Tags": [
            {
                "Key": "Name",
                "Value": "charan-nacl"
            }
        ],
        "VpcId": "vpc-03d3f1cd5780cbf6f",
        "OwnerId": "381492023386"
    },
    "ClientToken": "2a706638-fae6-4187-ba2e-afb615cff3eb"
}
ubuntu@ip-172-31-39-155:~$
```

Network ACLs (3) Info

Actions | Create network ACL

Find Network ACLs by attribute or tag

Name	Network ACL ID	Associated with	Default	VPC ID	Inbounc
–	acl-0c5d9bdce12d0060c	2 Subnets	Yes	vpc-03d3f1cd5780cbf6f / charan-vpc	2 Inbour
–	acl-0b62b660b7d872b92	6 Subnets	Yes	vpc-0778e7cff7ea8260d	2 Inbour
charan-nacl	acl-05bcd5635092a317d	–	No	vpc-03d3f1cd5780cbf6f / charan-vpc	1 Inbour

Add Outbound Rules (Crucial for NACL):

- Because NACLs are stateless, if you don't allow traffic **out**, the data can't return to the user. You must allow "Ephemeral Ports" (1024-65535).

`aws ec2 create-network-acl-entry --network-acl-id acl-id --egress --rule-number 100 --protocol tcp --port-range From=1024,To=65535 --cidr-block 0.0.0.0/0 --rule-action allow`

```
ubuntu@ip-172-31-39-155:~$ aws ec2 create-network-acl-entry --network-acl-id acl-05bcd5635092a317d --egress --rule-number 100 --protocol tcp --port-range Fr
om=1024,To=65535 --cidr-block 0.0.0.0/0 --rule-action allow
ubuntu@ip-172-31-39-155:~$
```

acl-05bcd5635092a317d / charan-nacl Actions

Details Info

Network ACL ID	Associated with	Default	VPC ID
acl-05bcd5635092a317d	–	No	vpc-03d3f1cd5780cbf6f / charan-vpc

Owner: 381492023386

Inbound rules | Outbound rules | Subnet associations | Tags

Outbound rules (2) Edit outbound rules

Filter outbound rules

Rule number	Type	Protocol	Port range	Destination	Allow/Deny
100	Custom TCP	TCP (6)	1024 - 65535	0.0.0.0/0	Allow
*	All traffic	All	All	0.0.0.0/0	Deny

7. ## Final Verification:

Run this command to verify all resources created with the **"charan-"** tag:

`aws ec2 describe-vpcs --filters "Name=tag:Name,Values=charan-*" --query "Vpcs[*].{ID:VpcId,CIDR:CidrBlock}" --output table`

```
ubuntu@ip-172-31-39-155:~$ aws ec2 describe-vpcs --filters "Name=tag:Name,Values=charan-*" --query "Vpcs[*].{ID:VpcId,CIDR:CidrBlock}" --output table
-----------------------------------------
|              DescribeVpcs             |
+--------------+------------------------+
|     CIDR     |           ID           |
+--------------+------------------------+
|  10.0.0.0/16 |  vpc-03d3f1cd5780cbf6f |
+--------------+------------------------+
ubuntu@ip-172-31-39-155:~$
```

VPCs | VPC encryption controls

Your VPCs (2) Info — Last updated 1 minute ago — Actions — Create VPC

Find VPCs by attribute or tag

Name	VPC ID	State	Encryption c...	Encryption control ...	Block Public...	IPv
charan-vpc	vpc-03d3f1cd5780cbf6f	Available	–	–	Off	10.
–	vpc-0778e7cff7ea8260d	Available	–	–	Off	172

8. ## Common Troubleshooting Connection Issues:

- If you cannot connect to your instance via SSH, check these three common points:
  1. **Internet Gateway (IGW) Attachment**: Ensure your IGW is not just created, but **attached** to your VPC.
     - *Check:* `aws ec2 describe-internet-gateways --filters "Name=attachment.vpc-id,Values=vpc-id"`

```
ubuntu@ip-172-31-39-155:~$ aws ec2 describe-internet-gateways --filters "Name=attachment.vpc-id,Values=vpc-03d3f1cd5780cbf6f"
{
    "InternetGateways": [
        {
            "Attachments": [
                {
                    "State": "available",
                    "VpcId": "vpc-03d3f1cd5780cbf6f"
                }
            ],
            "InternetGatewayId": "igw-0f71d8bf8fa0ff3c4",
            "OwnerId": "381492023386",
            "Tags": [
                {
                    "Key": "Name",
                    "Value": "charan-igw"
                }
            ]
        }
    ]
}
```

  2. **Public Route Table**: Confirm there is a route for `0.0.0.0/0` pointing to the `igw-id`. Without this, the subnet is effectively private.

- *Check:* `aws ec2 describe-route-tables --route-table-id rtb-id`

```
ubuntu@ip-172-31-39-155:~$ aws ec2 describe-route-tables --route-table-id rtb-0c4f3e3c0e9ba1681
{
    "RouteTables": [
        {
            "Associations": [
                {
                    "Main": false,
                    "RouteTableAssociationId": "rtbassoc-023f30d25c4d2062e",
                    "RouteTableId": "rtb-0c4f3e3c0e9ba1681",
                    "SubnetId": "subnet-0853f3e3d117a36fe",
                    "AssociationState": {
                        "State": "associated"
                    }
                }
            ],
            "PropagatingVgws": [],
            "RouteTableId": "rtb-0c4f3e3c0e9ba1681",
            "Routes": [
                {
                    "DestinationCidrBlock": "10.0.0.0/16",
                    "GatewayId": "local",
                    "Origin": "CreateRouteTable",
                    "State": "active"
                }
            ],
            "Tags": [
                {
                    "Key": "Name",
                    "Value": "charan-private-rt"
                }
            ],
            "VpcId": "vpc-03d3f1cd5780cbf6f",
            "OwnerId": "381492023386"
        }
    ]
}
ubuntu@ip-172-31-39-155:~$
```

```
ubuntu@ip-172-31-39-155:~$ aws ec2 describe-route-tables --route-table-id rtb-0b7d4e86d84a62660
{
    "RouteTables": [
        {
            "Associations": [
                {
                    "Main": false,
                    "RouteTableAssociationId": "rtbassoc-0b2951b8fecca9d7a",
                    "RouteTableId": "rtb-0b7d4e86d84a62660",
                    "SubnetId": "subnet-094379f0257f472b6",
                    "AssociationState": {
                        "State": "associated"
                    }
                }
            ],
            "PropagatingVgws": [],
            "RouteTableId": "rtb-0b7d4e86d84a62660",
            "Routes": [
                {
                    "DestinationCidrBlock": "10.0.0.0/16",
                    "GatewayId": "local",
                    "Origin": "CreateRouteTable",
                    "State": "active"
                },
                {
                    "DestinationCidrBlock": "0.0.0.0/0",
                    "GatewayId": "igw-0f71d8bf8fa0ff3c4",
                    "Origin": "CreateRoute",
                    "State": "active"
                }
            ],
            "Tags": [
                {
                    "Key": "Name",
                    "Value": "charan-public-rt"
                }
            ],
            "VpcId": "vpc-03d3f1cd5780cbf6f",
            "OwnerId": "381492023386"
        }
    ]
}
ubuntu@ip-172-31-39-155:~$
```

3. **Public IP Assignment**: Ensure the EC2 instance has a **Public IP** address. If you didn't enable "Auto-assign public IP" on the subnet, you must manually assign an **Elastic IP**.
   - *Check:* `aws ec2 describe-instances --instance-id i-id --query "Reservations[*].Instances[*].PublicIpAddress"`
4. If everything looks right but SSH still times out, double-check your **NACL** rules. Remember

that NACLs are **stateless**, so they need an outbound rule for ports **1024-65535** to let the SSH response back out!

9. ## Summary of Infrastructure:

   - **VPC**: `charan-vpc` is isolated and ready.
   - **Networking**: Internet Gateway (`charan-igw`) is attached to the Public Route Table.
   - **Security**: `charan-sg` is restricted to **Port 22 (Your IP)** and **Port 80 (Anywhere)**.
   - **Connectivity**: Instances launched in the **Public Subnet** will be reachable via SSH.

10. ## Cleanup (Optional):

    To avoid costs when finished, remember to delete resources in reverse order:

    1. Terminate Instances
    2. Delete Security Group
    3. Detach/Delete IGW
    4. Delete Subnets/VPC